

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA



PRÁCTICA N° 2

INF-133 PROGRAMACIÓN WEB 3

Node.js + Express + Sequelize + SQLite

Docente: Lic. Jhonny Felipez Andrade

Estudiante: Univ. Juan Luis Tijo Osco

CI: 699250 Ip

Gestión I/2026

Fecha: 02 de junio de 2026

Ejercicio 1

POST /categorias - Registrar nueva categoría

Endpoint que permite registrar una nueva categoría enviando nombre y descripción en el body de la petición.

Código del Controlador

```
// controllers/categoriaController.js
exports.createCategoria = async (req, res) => {
  try {
    const { nombre, descripcion } = req.body;
    const nueva = await Categoria.create({ nombre, descripcion });
    res.status(201).json(nueva);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

Ruta

```
router.post('/', categoriaController.createCategoria);
```

Prueba en Thunder Client

Método: POST

URL: http://localhost:3000/categorias

Body (JSON):

```
{
  "nombre": "Electrónica",
  "descripcion": "Dispositivos electrónicos y gadgets"
}
```

Respuesta esperada: 201 Created con los datos de la categoría creada.

Ejercicio 2

GET /categorias - Listar todas las categorías

Endpoint que devuelve todas las categorías registradas en la base de datos.

Código del Controlador

```
// controllers/categoriaController.js
exports.getAllCategorias = async (req, res) => {
  try {
    const categorias = await Categoria.findAll();
    res.json(categorias);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

Ruta

```
router.get('/', categoriaController.getAllCategorias);
```

Prueba en Thunder Client

Método: GET

URL: <http://localhost:3000/categorias>

Respuesta esperada: 200 OK con un arreglo de todas las categorías.

Ejercicio 3

GET /categorias/:id - Obtener categoría con productos

Endpoint que devuelve la categoría con el ID especificado e incluye todos los productos que pertenecen a esa categoría.

Código del Controlador

```
// controllers/categoriaController.js
exports.getCategoriaById = async (req, res) => {
  try {
    const { id } = req.params;
    const categoria = await Categoria.findByPk(id, {
      include: [{ model: Producto }]
    });
    if (!categoria)
      return res.status(404).json({ message: 'Categoría no encontrada' });
    res.json(categoria);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

Relación en modelos/index.js

```
Categoria.hasMany(Producto, {
  foreignKey: 'categoriaId',
  onDelete: 'CASCADE'
});
Producto.belongsTo(Categoria, {
  foreignKey: 'categoriaId'
});
```

Ruta

```
router.get('/:id', categoriaController.getCategoriaById);
```

Prueba en Thunder Client

Método: GET

URL: http://localhost:3000/categorias/1

Respuesta esperada: 200 OK con la categoría y su arreglo de productos.

Ejercicio 4

PATCH /categorias/:id - Actualizar categoría

Endpoint que permite actualizar todos los datos de la categoría con el ID especificado.

Código del Controlador

```
// controllers/categoriaController.js
exports.updateCategoria = async (req, res) => {
  try {
    const { id } = req.params;
    const { nombre, descripcion } = req.body;
    const categoria = await Categoria.findByPk(id);
    if (!categoria)
      return res.status(404).json({ message: 'Categoría no encontrada' });
    await categoria.update({ nombre, descripcion });
    res.json(categoria);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

Ruta

```
router.patch('/:id', categoriaController.updateCategoria);
```

Prueba en Thunder Client

Método: PATCH

URL: http://localhost:3000/categorias/1

Body (JSON):

```
{
  "nombre": "Electrónica Actualizada",
  "descripcion": "Nueva descripción"
}
```

Respuesta esperada: 200 OK con los datos actualizados de la categoría.

Ejercicio 5

DELETE /categorias/:id - Eliminar categoría y sus productos

Endpoint que elimina la categoría indicada y, automáticamente, elimina todos los productos que pertenecen a esa categoría mediante CASCADE.

Código del Controlador

```
// controllers/categoriaController.js
exports.deleteCategoria = async (req, res) => {
  try {
    const { id } = req.params;
    const categoria = await Categoria.findByPk(id);
    if (!categoria)
      return res.status(404).json({ message: 'Categoría no encontrada' });
    await categoria.destroy();
    res.json({ message: 'Categoría y productos asociados eliminados correctamente' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

CASCADE en modelos/index.js

```
Categoria.hasMany(Producto, {
  foreignKey: 'categoriaId',
  onDelete: 'CASCADE' // Elimina productos automáticamente
});
```

Ruta

```
router.delete('/:id', categoriaController.deleteCategoria);
```

Prueba en Thunder Client

Método: DELETE

URL: http://localhost:3000/categorias/1

Respuesta esperada: 200 OK con mensaje de confirmación. Los productos asociados se eliminan automáticamente por el CASCADE.

Anexo - Código Completo del Proyecto

package.json

```
{
  "name": "practica-2-node-express",
  "version": "1.0.0",
  "description": "Práctica N° 2 - INF-133",
  "main": "app.js",
  "dependencies": {
    "docx": "0.7.1",
    "express": "^4.18.2",
    "sequelize": "^6.31.0",
    "sql.js": "1.14.1"
  },
  "scripts": {
    "start": "node app.js"
  }
}
```

config/db.js

```
const { Sequelize } = require('sequelize');
const sqliteAdapter = require('./sqljs-adapter');

const sequelize = new Sequelize({
  dialect: 'sqlite',
  storage: './database.sqlite',
  logging: false,
  dialectModule: sqliteAdapter
});

module.exports = sequelize;
```

models/Categoria.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/db');

const Categoria = sequelize.define('Categoria', {
  nombre: {
    type: DataTypes.STRING,
    allowNull: false
  },
  descripcion: {
    type: DataTypes.STRING
  }
}, {
  tableName: 'categorias'
});

module.exports = Categoria;
```

models/Producto.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/db');

const Producto = sequelize.define('Producto', {
  nombre: {
    type: DataTypes.STRING,
    allowNull: false
  },
  precio: {
    type: DataTypes.DECIMAL(10, 2),
    allowNull: false
  }
}, {
  tableName: 'productos'
});

module.exports = Producto;
```

models/index.js

```
const sequelize = require('../config/db');
const Categoria = require('./Categoria');
```

```

const Producto = require('./Producto');

// Relaciones
Categoria.hasMany(Producto, {
  foreignKey: 'categoriaId',
  onDelete: 'CASCADE' // Punto 5: Eliminar productos automáticamente
});
Producto.belongsTo(Categoria, {
  foreignKey: 'categoriaId'
});

module.exports = {
  sequelize,
  Categoria,
  Producto
};

```

controllers/categoriaController.js

```

const { Categoria, Producto } = require('../models');

// 1. POST /categorias - Registrar una nueva categoría
exports.createCategoria = async (req, res) => {
  try {
    const { nombre, descripcion } = req.body;
    const nuevaCategoria = await Categoria.create({ nombre, descripcion });
    res.status(201).json(nuevaCategoria);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// 2. GET /categorias - Devolver todas las categorías
exports.getAllCategorias = async (req, res) => {
  try {
    const categorias = await Categoria.findAll();
    res.json(categorias);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// 3. GET /categorias/:id - Devolver categoría por ID e incluir productos
exports.getCategoriaById = async (req, res) => {
  try {
    const { id } = req.params;
    const categoria = await Categoria.findByPk(id, {
      include: [{ model: Producto }]
    });
    if (!categoria) {
      return res.status(404).json({ message: 'Categoría no encontrada' });
    }
    res.json(categoria);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// 4. PATCH /categorias/:id - Actualizar todos los datos
exports.updateCategoria = async (req, res) => {
  try {
    const { id } = req.params;
    const { nombre, descripcion } = req.body;
    const categoria = await Categoria.findByPk(id);

    if (!categoria) {
      return res.status(404).json({ message: 'Categoría no encontrada' });
    }

    await categoria.update({ nombre, descripcion });
    res.json(categoria);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// 5. DELETE /categorias/:id - Eliminar categoría y sus productos (Cascade)
exports.deleteCategoria = async (req, res) => {
  try {
    const { id } = req.params;
    const categoria = await Categoria.findByPk(id);

    if (!categoria) {
      return res.status(404).json({ message: 'Categoría no encontrada' });
    }
  }
};

```



```

    // Al tener onDelete: 'CASCADE' definido en el modelo,
    // Sequelize (o la DB) eliminará los productos asociados.
    await categoria.destroy();
    res.json({ message: 'Categoría y productos asociados eliminados correctamente' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

```

routes/categoriaRoutes.js

```

const express = require('express');
const router = express.Router();
const categoriaController = require('../controllers/categoriaController');

router.post('/', categoriaController.createCategoria);
router.get('/', categoriaController.getAllCategorias);
router.get('/:id', categoriaController.getCategoriaById);
router.patch('/:id', categoriaController.updateCategoria);
router.delete('/:id', categoriaController.deleteCategoria);

module.exports = router;

```

app.js

```

const express = require('express');
const { sequelize, Categoria, Producto } = require('./models');
const categoriaRoutes = require('./routes/categoriaRoutes');

const app = express();
app.use(express.json());

// Rutas
app.use('/categorias', categoriaRoutes);

const PORT = 3000;

// Sincronizar Base de Datos e Iniciar Servidor
sequelize.sync({ force: true }).then(async () => {
  console.log('Base de datos sincronizada.');
```

```

  // iniciales según el requerimiento
  await Categoria.bulkCreate([
    { nombre: 'Electrónica', descripcion: 'Dispositivos electrónicos y gadgets' },
    { nombre: 'Oficina', descripcion: 'Material y accesorios de oficina' }
  ]);

  // Agregar un producto de ejemplo para probar el punto 3 y 5
  const cat = await Categoria.findOne({ where: { nombre: 'Electrónica' } });
  await Producto.create({ nombre: 'Laptop', precio: 1200, categoriaId: cat.id });

  app.listen(PORT, () => {
    console.log(`Servidor corriendo en http://localhost:${PORT}`);
  });
});

```